

Multimodal Generative AI Models

Designing and Implementing Vision and Language Models

Alessandro Suglia



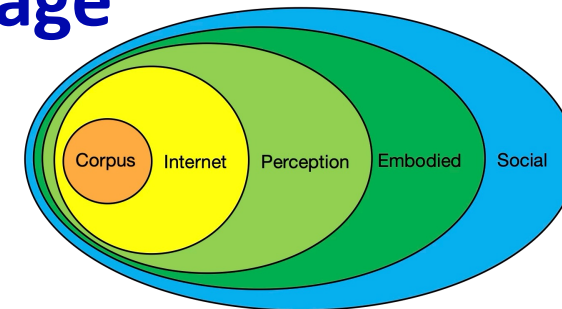
Outline

Aim: Understand how frontier VLMs are designed, implemented and trained

Outline:

- Recap of why we care about Perceptually-grounded NLP models
- Training pipeline for VLMs
- SmolVLM: Deep-dive into Visual prefix VLMs
- An overview of the SOTA open weights ecosystem
- Future avenues on VLM research

Recap: Experience Grounds Language

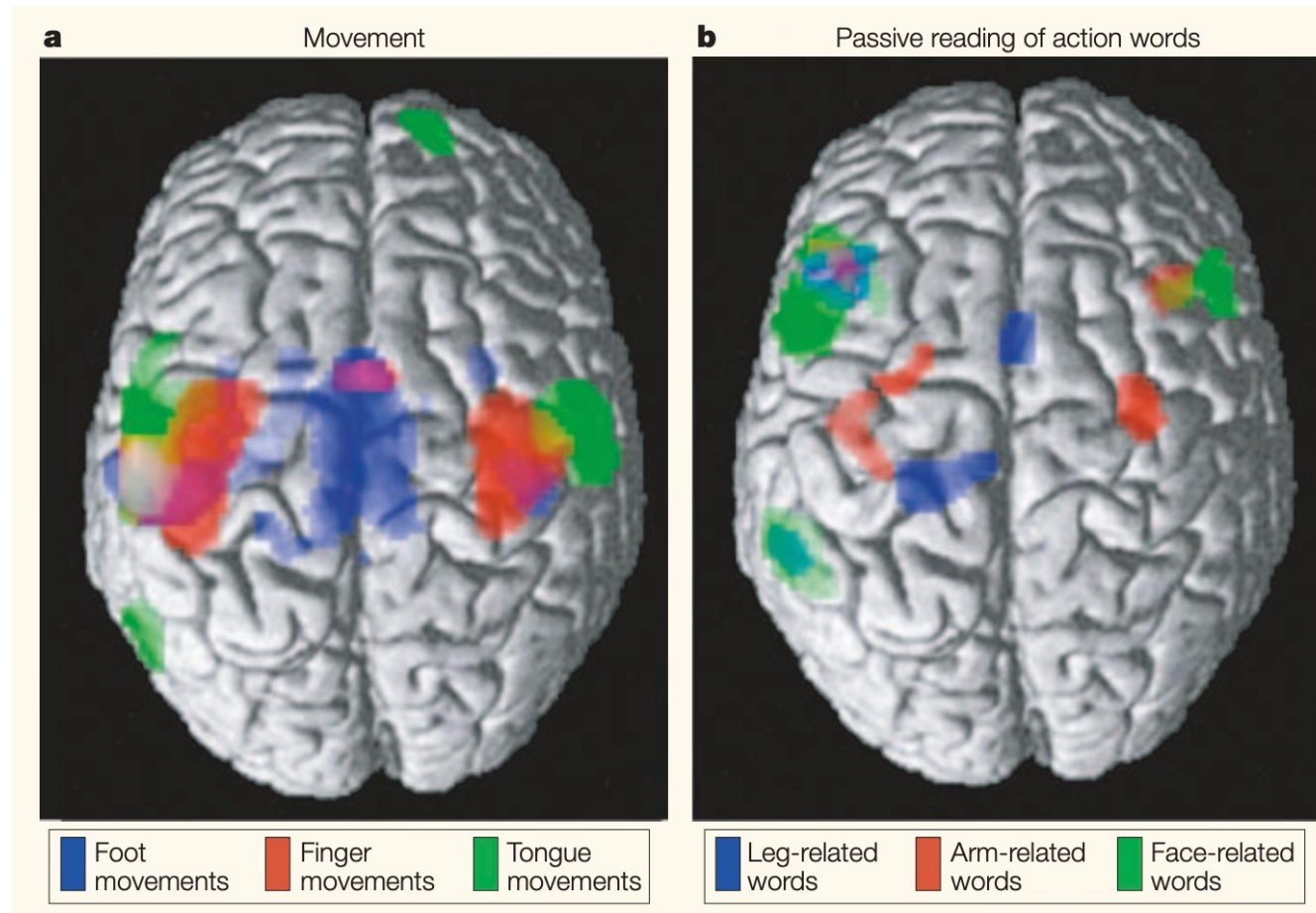


You can't learn language:

- **from the radio (Internet).** $WS_2 \subset WS_3$
 - A task learner cannot be said to be in WS_3 if it can succeed without perception (e.g., visual, auditory).
- **from the television.** $WS_3 \subset WS_4$
 - A task learner cannot be said to be in WS_4 if the space of its world actions and consequences can be enumerated.
- **by yourself.** $WS_4 \subset WS_5$
 - A task learner cannot be said to be in WS_5 unless achieving its goals requires cooperating with a human in the loop.

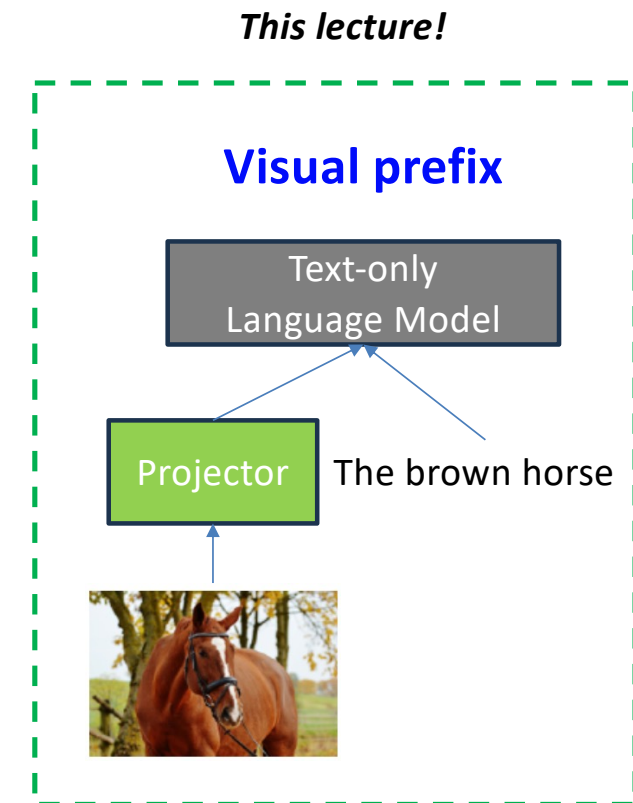
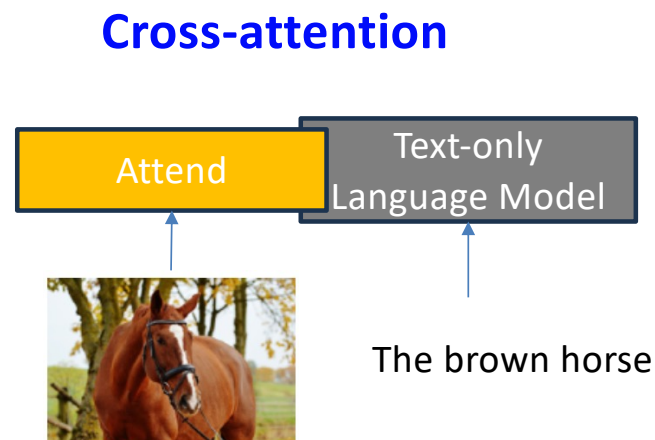
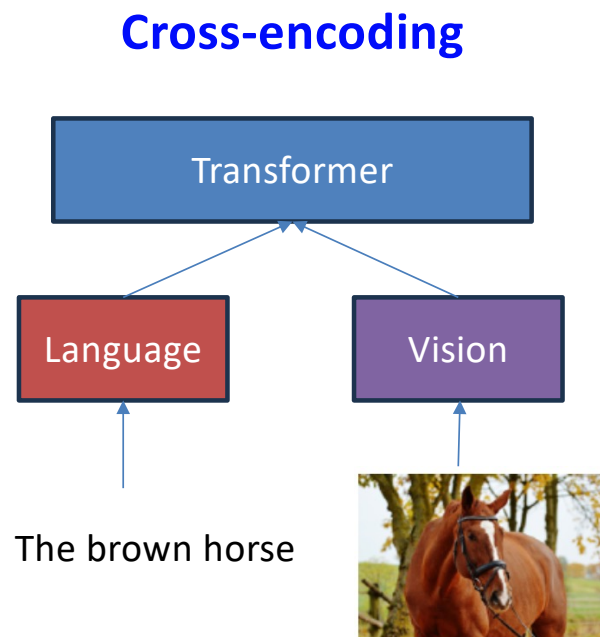
Bisk, Y., Holtzman, A., Thomason, J., Andreas, J., Bengio, Y., Chai, J., ... & Turian, J. (2020, November). Experience Grounds Language. EMNLP 2020

Recap: Evidence in favour of Multimodal/Grounded NLP



Pulvermüller, F. (2005). Brain mechanisms linking language and action. *Nature reviews neuroscience*, 6(7), 576-582.

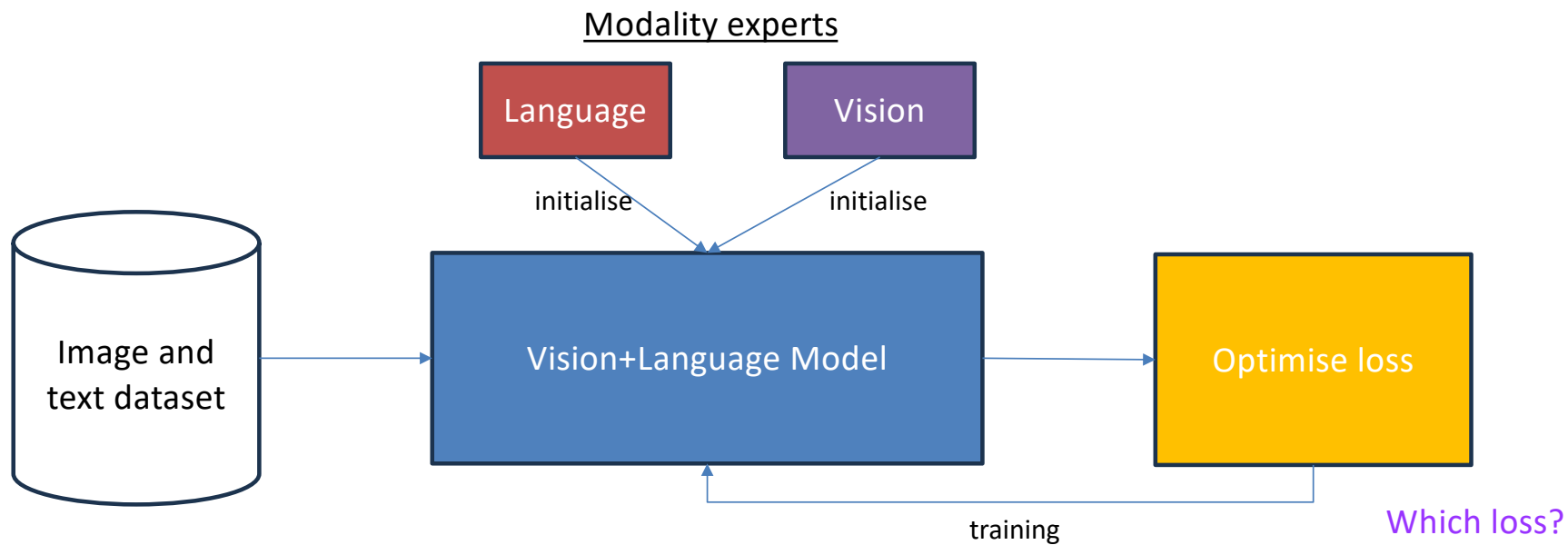
Main Approaches for Designing VLMs



Cross-encoding: Bugliarello et al. 2021. Multimodal Pretraining Unmasked: A Meta-Analysis and a Unified Framework of Vision-and-Language BERTs. TACL.

Cross-attention: Alayrac, Jean-Baptiste, et al. "Flamingo: a visual language model for few-shot learning." NeurIPS (2022).

Training pipeline

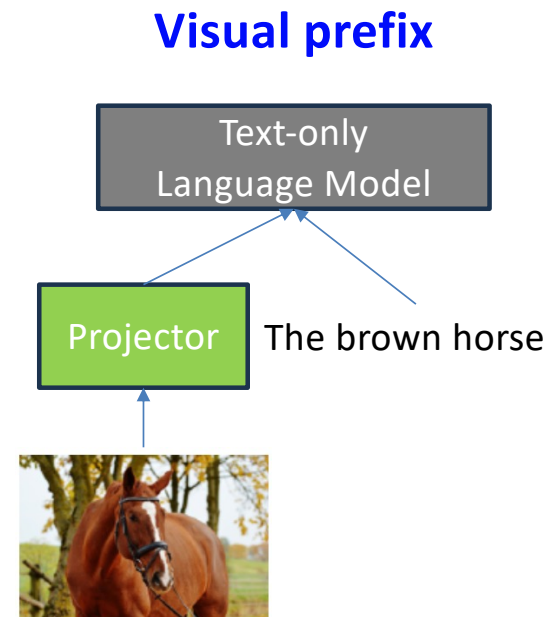


Deep-dive into visual prefix-based VLMs

DESIGNING STATE-OF-THE-ART VISION AND LANGUAGE MODELS

Visual prefix-based VLMs – The intuition

- Assume that you have access to two modality experts:
 - The **language expert** = LLM
 - The **vision expert** = Pretrained Vision Transformer
- Use the pre-trained LLM as the backbone architecture
- Design a **projector module** that converts the representation of the vision expert so that they are compatible with the LLM
- Train the language model to generate tokens via **cross-entropy loss**
 - The same loss we use for training text-only LLMs



Visual prefix-based VLMs – The intuition

- Assume that you have access to two modality experts:
 - The language expert
 - The vision expert
- Use the pre-trained Transformer architecture
- Design a projected representation so that they are comparable
- Train the language model to generate tokens via cross-entropy loss
 - The same loss we use for training text-only LLMs

This is the model design adopted by most model providers!

Why prefix?

prefix

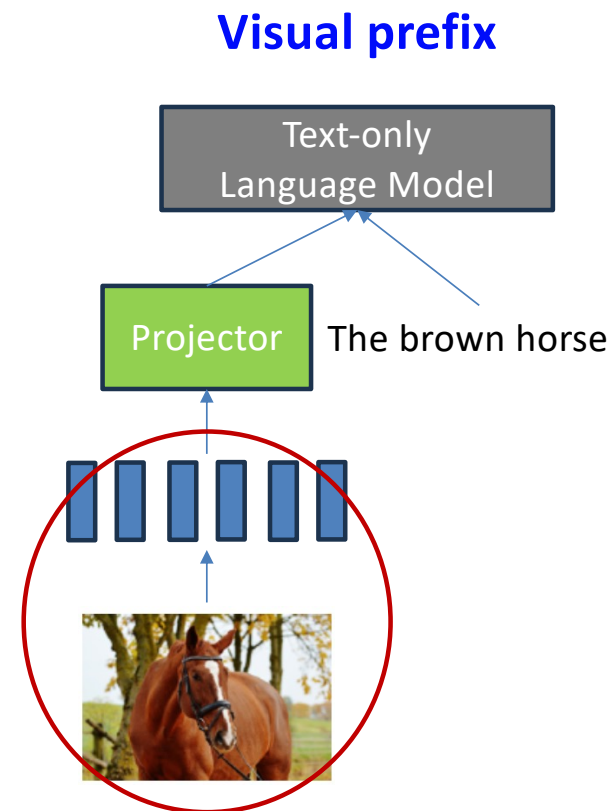
only
Model

brown horse

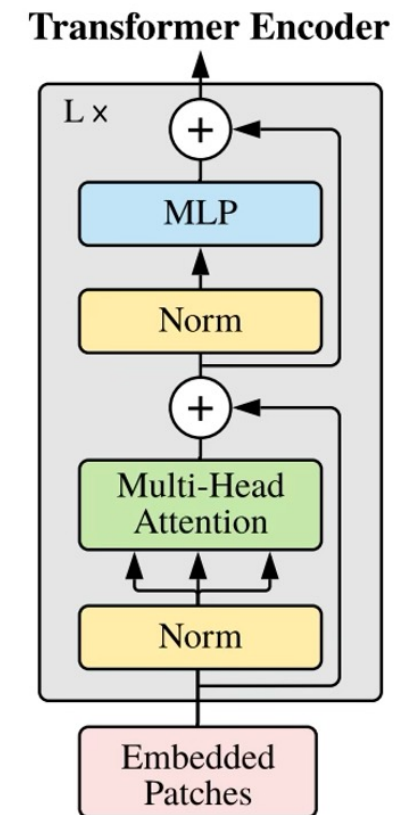
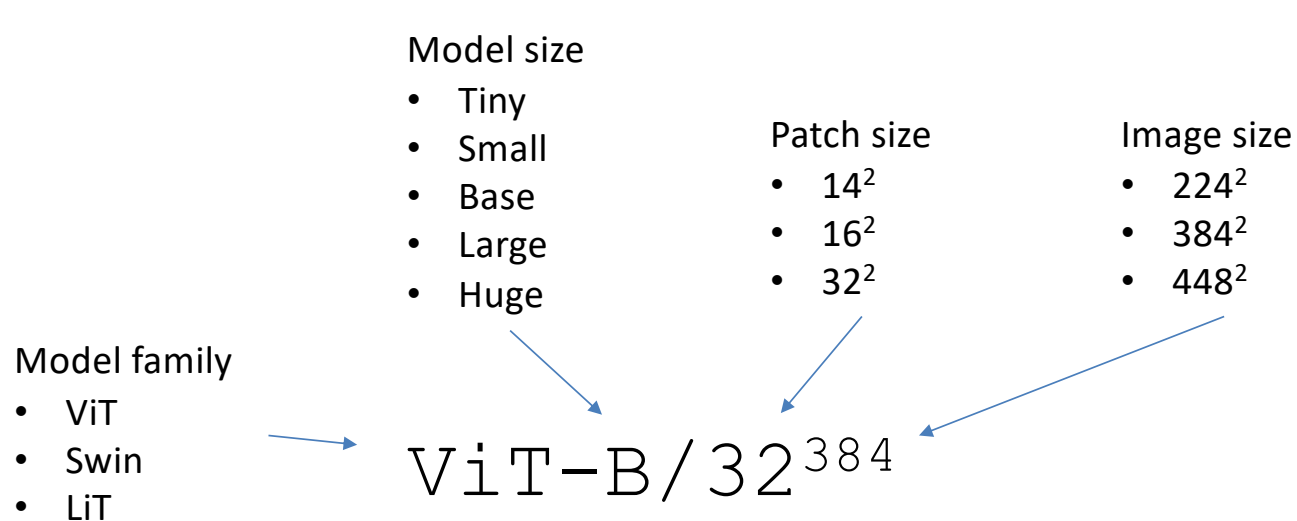


The visual encoder

- This is typically a pretrained Vision Transformer
- Pretraining can be performed in several ways:
 - Vision only: using reconstruction loss
 - Vision+Language: using contrastive loss
- Generally models trained with V+L data have superior performance compared to Vision only models
- This model *typically uses* a fixed number of visual tokens that represents a single image



Common Pretrained Visual Encoders



Number of patches in Pretrained Visual Encoders

The Vision Transformer always extract non-overlapping square patches of size $P \times P$ from an image of size $W \times H$. This will be the “tokens” that will be processed by the VLM!

Let's assume a square image and compute the number of patches:

Image Size (H×H)	Patch Size (P×P)	Number of Patches ($NP=(H/P)^2$)
224 × 224	16 × 16	$(224/16)^2 = 14^2 = \mathbf{196}$
224 × 224	32 × 32	$(224/32)^2 = 7^2 = \mathbf{49}$
224 × 224	8 × 8	$(224/8)^2 = 28^2 = \mathbf{784}$
384 × 384	16 × 16	$(384/16)^2 = 24^2 = \mathbf{576}$

Number of patches in Pretrained Visual Encoders

The Vision Transformer always extract non-overlapping square patches of size $P \times P$ from an image of size $W \times H$. This will be the “tokens” that will be processed by the VLM!

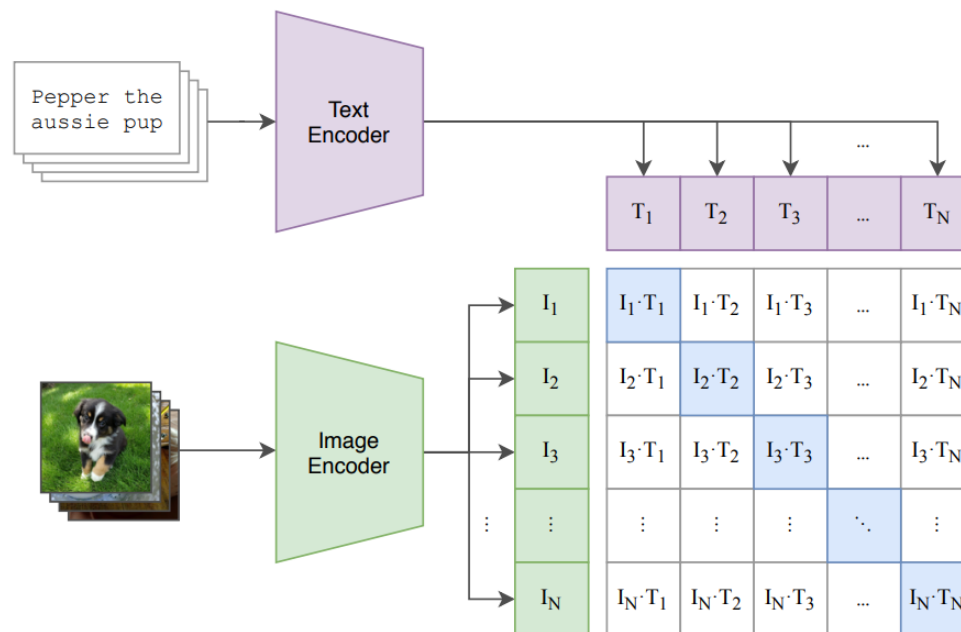
Let's assume a

The higher the number of patches, the higher the memory required to integrate a vision encoder in an LLM!

Image			
			196
			49
			784
	384×384	16×16	$(384/16)^2 = 24^2 = 576$

Common Pretrained Visual Contrastive Encoders

- Combines a vision encoder with a language model
- It is trained with image-caption pairs datasets
- The objective is a contrastive loss that forces the model to bring **similar embeddings close** and **dissimilar ones far apart**
- Several SOTA variants:
 - OpenAI CLIP
 - Google SigLIP

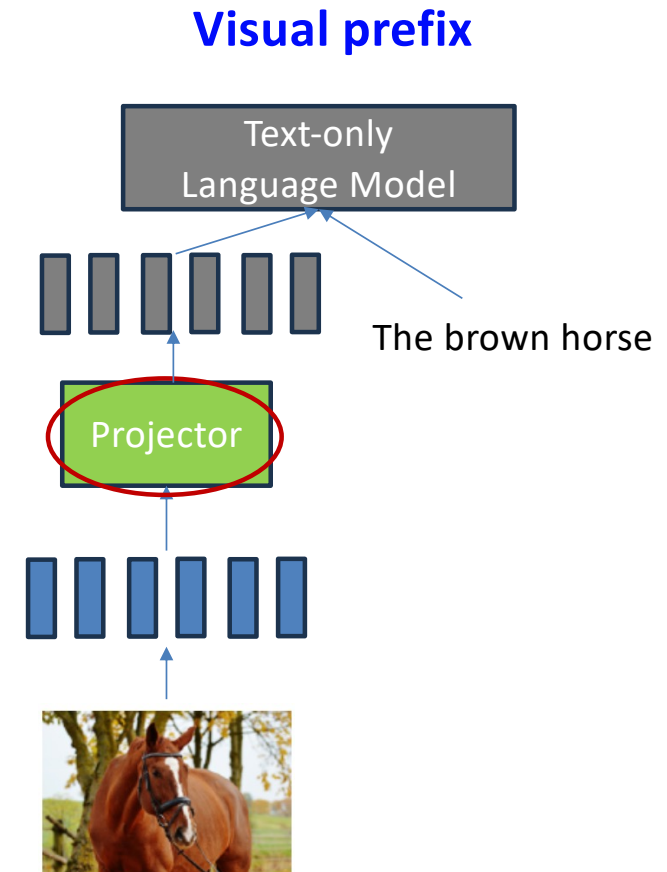


Zhai, Xiaohua, et al. "Sigmoid loss for language image pre-training." Proceedings of the IEEE/CVF international conference on computer vision. 2023.

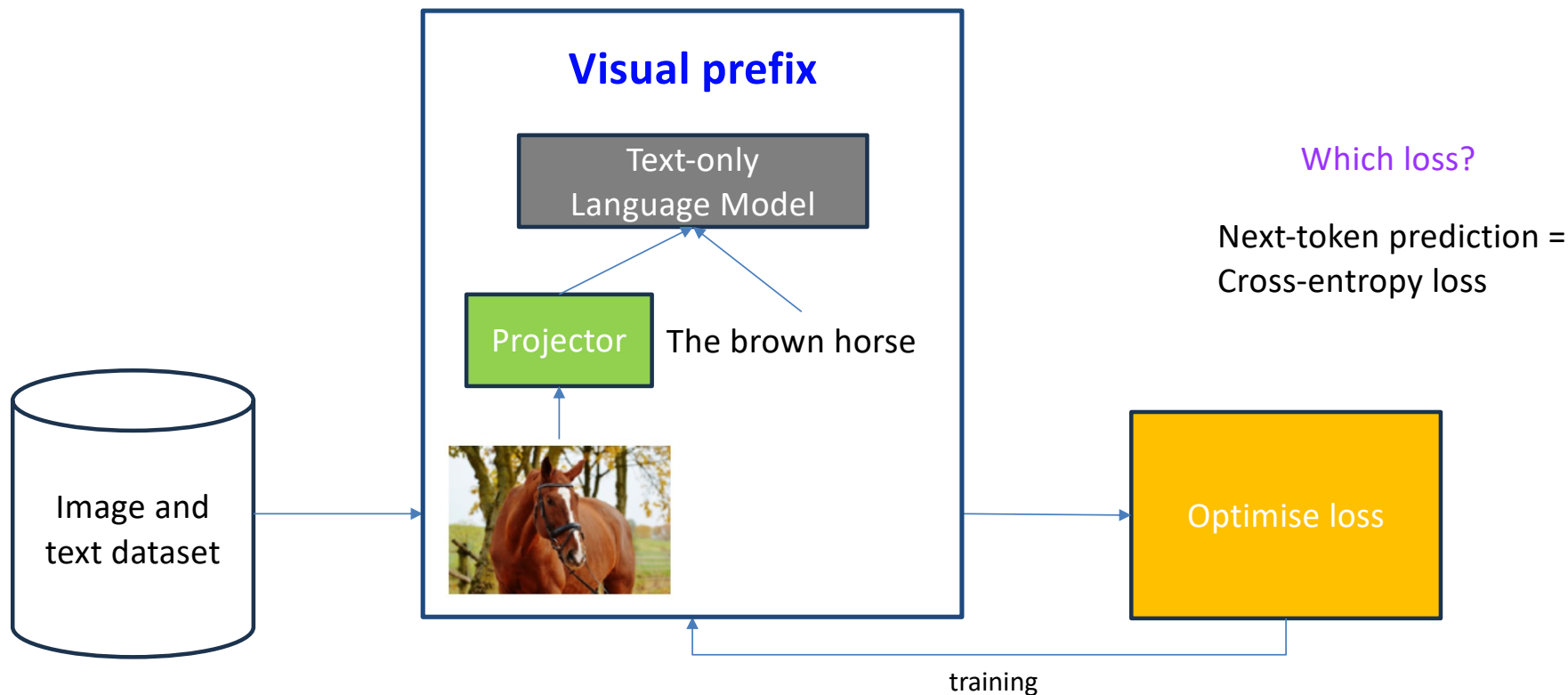
Radford, Alec, et al. "Learning transferable visual models from natural language supervision." International conference on machine learning. PmlR, 2021.

The projector

- The Vision encoder represents an image with a fixed number of **visual tokens**
- The hidden dimensionality of these tokens **is different** from the hidden size accepted by the text-only LLM
- How do we adapt them?
 - We need a **projector layer** to map the vision space to the language space
 - This turns visual tokens into tokens that the **LLM can encode**
- How do we implement it?
 - A feedforward neural network
 - Cross attention with learnable queries
 - ATNLP Tutorial 4 will explore other approaches!



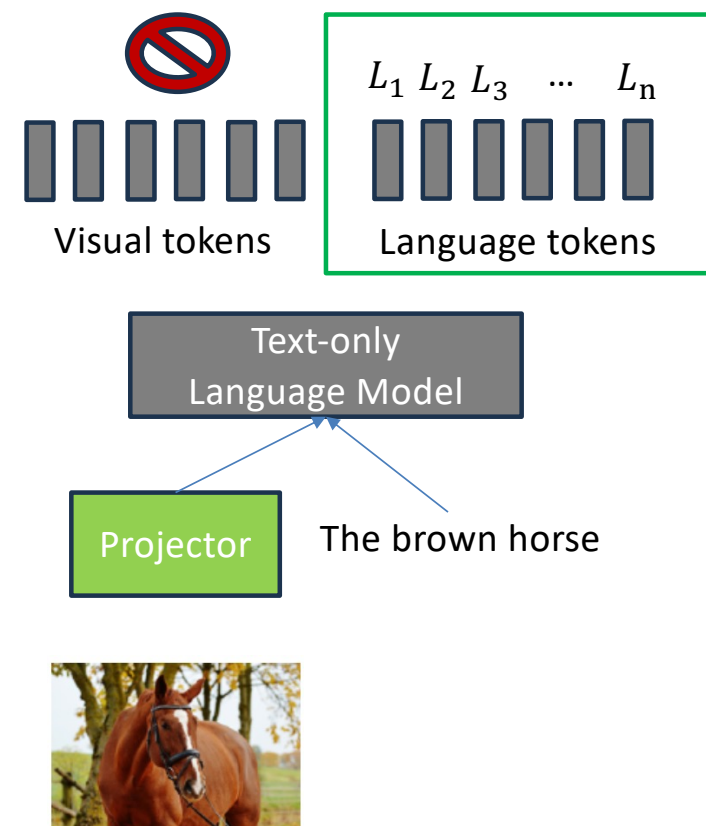
Training pipeline



The cross-entropy loss computation for VLMs

Optimise
loss

- We compute the cross-entropy loss only on the language tokens just like in a traditional LLM
- The ground truth values are typically the text tokens associated with the vision modality (e.g., caption)
- Predictions made for the visual tokens are discarded!
 - Why? We only have supervision for language tokens and none for visual tokens!



How do we train these models?

IMPLEMENTING STATE-OF-THE-ART VISION AND LANGUAGE MODELS

Implementing VLMs from the ground up

- We now have a good understanding of all the components that are part of the main Vision and Language models based on the visual prefix design. However, we still don't know how to implement them!
- We need to answer the following questions:
 - Data:
 - What datasets do we use?
 - What tasks do we teach the model to solve?
 - Model:
 - What vision encoder?
 - What projector?
 - What LLM?
 - Training:
 - What loss?
 - Do we train all the components?
 - Or do we freeze some?



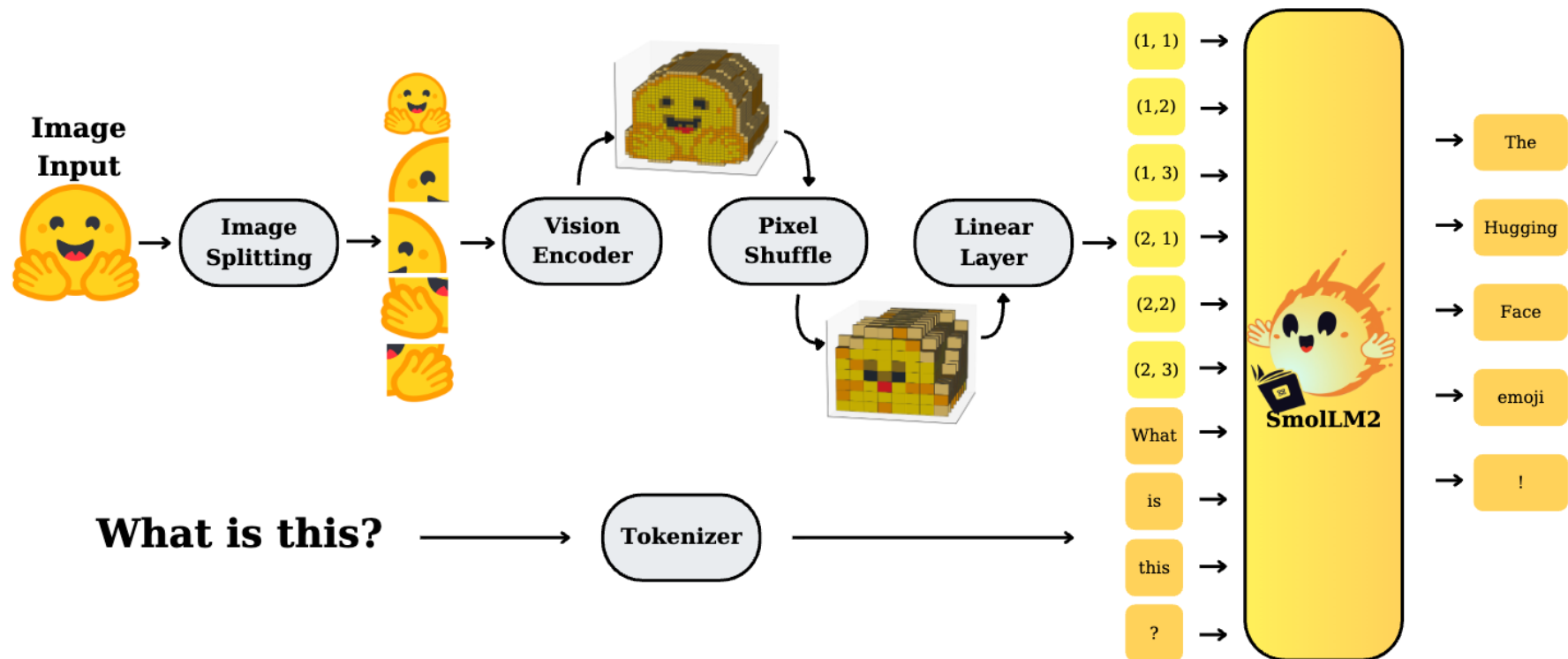
Huggingface SmolVLM

- In this lecture, we will focus our attention on [SmolVLM](#), an open source and open weight VLM
- Why should we care about open source and open weight models?
 - “*Closed AI Models Make Bad Baselines!*” (A. Rogers)
 - This is important to ensure reproducibility and progress
 - Facilitate adaptation and usage in specific downstream application scenarios
- We will study each component of the training pipeline:
 - Reference training dataset
 - Reference model architecture and training regime
 - Evaluation setup

<https://hackingsemantics.xyz/2023/closed-baselines/>

Marafioti, Andrés, et al. "Smolvlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

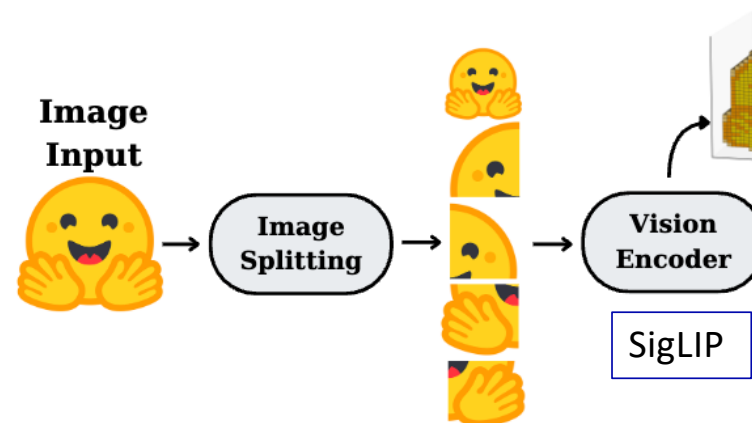
SmolVLM – Overview of the Model Architecture



Marafioti, Andrés, et al. "Smolvlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

SmolVLM – Representing Images and Videos

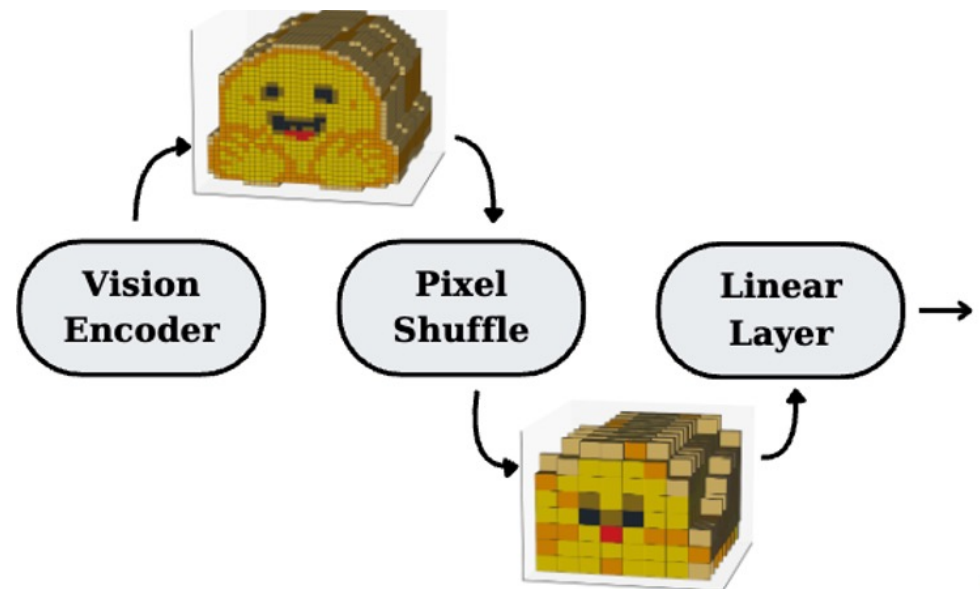
- The image processor is responsible for taking an image input and transforming it into tensors that can be digested by the vision encoder
 - SmolVLM uses SigLIP!
- Depending on the image resolution, the image processor **might split the image** into multiple sub-regions that are independently encoded by the vision encoder
- The full resolution image is then downsized to the vision encoder supported resolution
- The vision encoder generates visual tokens for each image split separately
- Video are supported as well: frames are sampled from a video and encoded as separate images



Marafioti, Andrés, et al. "SmolVlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

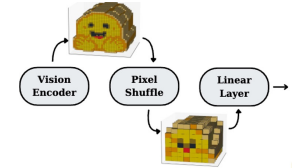
SmolVLM – Pixel Shuffle

- Early methods encode every pixel or patch individually, resulting in lengthy sequences—196 tokens for a 224×224 image at 16×16 resolution
- Compression methods are required to reduce the visual token count and minimise memory consumption
- **Pixel-shuffle**: allows to reduce token counts fourfold while maintaining finegrained visual understanding capabilities (e.g., OCR capability)
- This operation makes sure to reduce spatial resolution and increase representational density (along the depth dimension!)

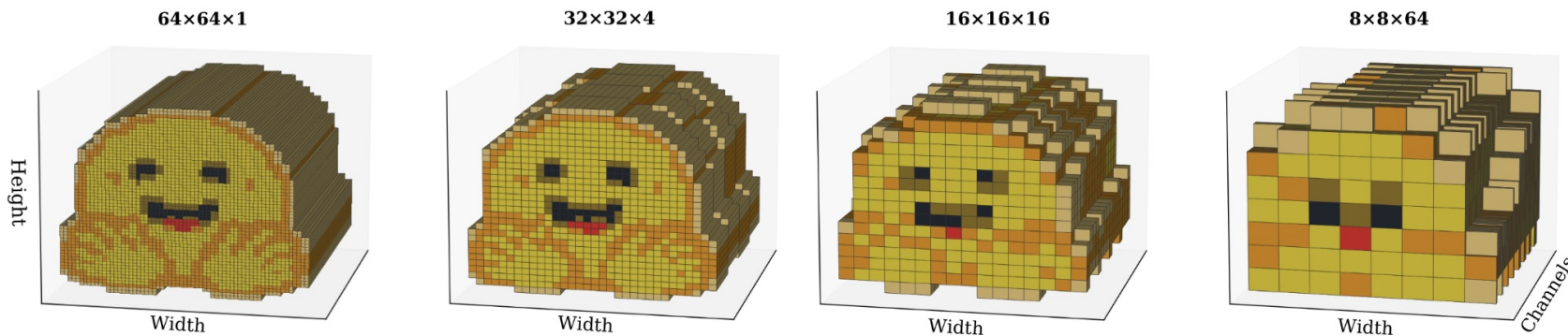


Marafioti, Andrés, et al. "Smolvlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

SmolVLM – Pixel Shuffle [2/3]

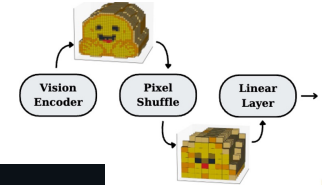


- Pixel shuffling is an operation that is applied to the features generated by the vision encoder
- In this way, pixel shuffling allows to compress *slices* of the feature space via an MLP that merges them
- This operation makes sure to reduce spatial resolution and increase representational density (along the depth dimension!)
- Pixel shuffle depends on the **shuffle ratio** r , a parameter that trade-offs compression and spatial fidelity; SmoVLM uses $r = 4$ to reduce visual token count which is essential for small models



Marafioti, Andrés, et al. "Smolvlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

SmolVLM – Pixel Shuffle [3/3]



```
class Idefics3Connector(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.scale_factor = config.scale_factor
        self.modality_projection = Idefics3SimpleMLP(config)

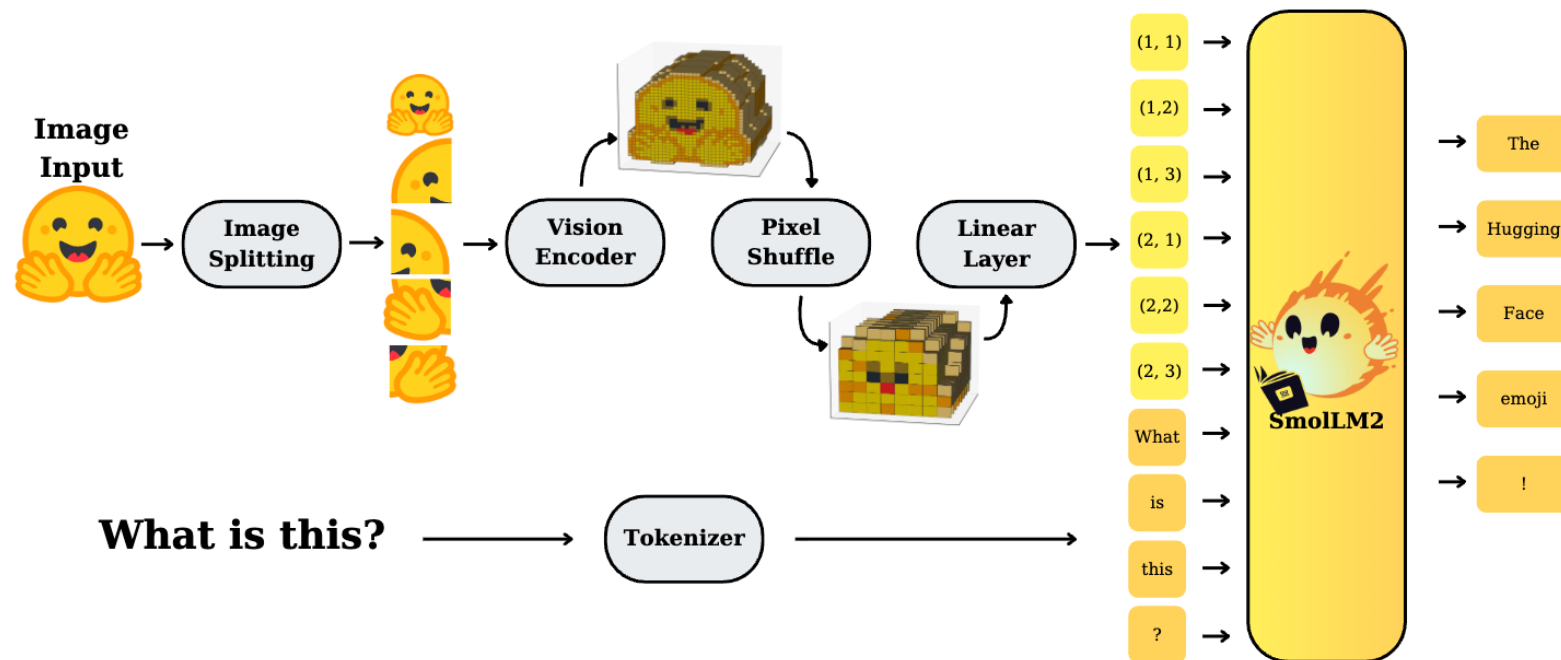
    def pixel_shuffle(self, x, scale_factor=2):
        bsz, seq, embed_dim = x.size()
        height = width = int(seq**0.5)
        x = x.view(bsz, height, width, embed_dim)
        x = x.view(bsz, height, int(width / scale_factor), embed_dim * scale_factor)
        x = x.permute(0, 2, 1, 3)
        x = x.reshape(bsz, int(width / scale_factor), int(height / scale_factor), embed_dim * (scale_factor**2))
        x = x.permute(0, 2, 1, 3)
        x = x.reshape(bsz, int(seq / (scale_factor**2)), embed_dim * (scale_factor**2))
        return x

    def forward(self, image_hidden_states):
        image_hidden_states = self.pixel_shuffle(image_hidden_states, self.scale_factor)
        image_hidden_states = self.modality_projection(image_hidden_states)
        return image_hidden_states
```

Pixel shuffling originally proposed in Idefics3; Code snippet from:
https://github.com/huggingface/transformers/blob/v4.57.0/src/transformers/models/idefics3/modeling_idefics3.py

SmolVLM – Visual prefix design

Based on the visual prefix design, we leverage the SmolLM2 pretrained weights as a backbone architecture for our model



Marafioti, Andrés, et al. "Smolvlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

SmolVLM – Overview of the Training regime

- SmolVLM is a model able to encode both single image as well as video inputs (i.e., multiple image frames)
- Model training proceeds in **three stages** requiring specialised datasets:
 - **Adaptation stage**: required to teach the projector layer to adapt the vision encoder representations to the language model backbone
 - **Image stage**: required to distil image understanding abilities across a variety of different image formats
 - **Video stage**: required to enable understanding of multi-image content going from single and multiple images, as well video

Marafioti, Andrés, et al. "SmolVlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

SmolVLM – Adaptation stage

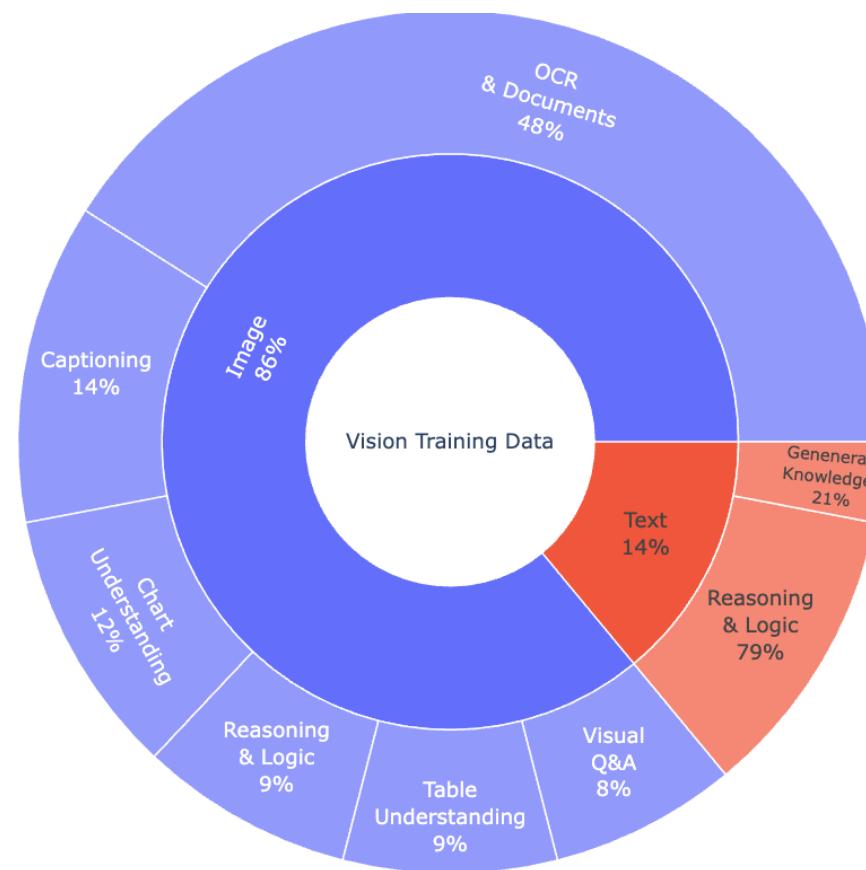
- The first stage of training is required to teach the projector layer to adapt the vision encoder representations to the language model backbone:
 - Vision encoder = SigLIP
 - Language model = SmolLM2
- They trained the projector layer and applied LORA on the vision encoder. This was done on mostly noisy web data and captioning
- In this case, we want to keep the language model frozen to make sure that we only learn an effective projector that knows how to adapt the visual representations to match the LLM backbone
- This stage is inspired by previous work on Llava (Liu et al. 2023) and IDEFICS-3 (Laurençon et al. 2024)

Liu, Haotian, et al. "Visual instruction tuning." NeurIPS 2023.

Laurençon, Hugo, et al. "What matters when building vision-language models?." NeurIPS 2024.

SmolVLM – Image stage

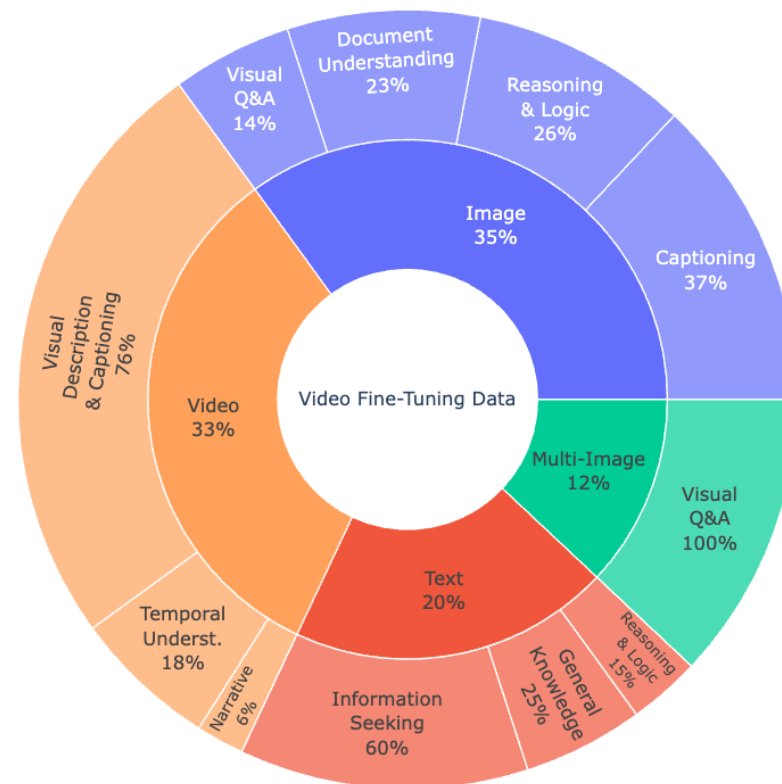
- The main objective of this stage is to further improve the model's ability to understand different image formats with a focus on OCR data (e.g., PDF, charts, tables, etc.)
- This is completed by finetuning the entire model (both the vision and LLM backbone) on a specific mixture of datasets reported in the graph
- A small proportion of textual data is also available to preserve the original instruction following abilities of the model



Marafioti, Andrés, et al. "SmolVlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

SmolVLM – Video stage

- The 3rd stage requires adapting the model to understand multi-image and video content.
- This is done in a separate stage as a form of curriculum learning (image -> multi-image -> video)
- Again, the entire model is trained on a careful mixture of dataset that includes image, multi-image, video and text only
 - Data is one of the key details that power all the models out there!



Marafioti, Andrés, et al. "SmolVlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

SmolVLM – Trained Models

- **SmolVLM-256M**: combines the 93M SigLIP-B/16 and the SmolLM2-135M (Allal et al., 2025). Operating on < 1GB GRAM makes it ideal for resource-constrained edge applications.
- **SmolVLM-500M**: A mid-range model with the same 93M SigLIP-B/16 paired with the larger SmolLM2-360M. Balancing memory efficiency and performance, it is suitable for moderate-resource edge devices.
- **SmolVLM-2.2B**: The largest variant, with a 400M SigLIP-SO400M and a 1.7B-parameter SmolLM2 backbone. This model maximizes performance while remaining deployable on higher-end edge systems.

Marafioti, Andrés, et al. "Smolvlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

SmolVLM – Large Scale Evaluation using VLMEvalKit

Capability Benchmark		SmolVLM 256M	SmolVLM 500M	SmolVLM 2.2B	Efficient OS
Single-Image	OCRBench (Liu et al., 2024e) Character Recognition	52.6%	61.0%	72.9%	54.7% MolmoE-A1B-7B
	AI2D (Kembhavi et al., 2016) Science Diagrams	46.4%	59.2%	70.0%	71.0% MolmoE-A1B-7B
	ChartQA (Masry et al., 2022) Chart Understanding	55.6%	62.8%	68.7%	48.0% MolmoE-A1B-7B
	TextVQA (Singh et al., 2019) Text Understanding	50.2%	60.2%	73.0%	61.5% MolmoE-A1B-7B
	DocVQA (Mathew et al., 2021) Document Understanding	58.3%	70.5%	80.0%	77.7% MolmoE-A1B-7B
	ScienceQA (Lu et al., 2022) High-school Science	73.8%	80.0%	89.6%	87.5% MolmoE-A1B-7B
Multi-task	MMMU (Yue et al., 2024a) College-level Multidiscipline	29.0%	33.7%	42.0%	33.9% MolmoE-A1B-7B
	MathVista (Lu et al., 2024b) General Math Understanding	35.9%	40.1%	51.5%	37.6% MolmoE-A1B-7B
	MMStar (Chen et al., 2024a) Multidisciplinary Reasoning	34.6%	38.3%	46.0%	43.1% MolmoE-A1B-7B
Video	Video-MME (Fu et al., 2024) General Video Understanding	33.7%	42.2%	52.1%	45.0% InternVL2-2B
	MLVU (Zhou et al., 2024) MovieQA + MSRVTTCap	40.6%	47.3%	55.2%	48.2% InternVL2-2B
	MVBench (Li et al., 2024b) Multiview Reasoning	32.7%	39.7%	46.3%	60.2% InternVL2-2B
	WorldSense (Hong et al., 2025) Temporal + Physics	29.7%	30.6%	36.2%	32.4% Qwen2VL-7B
	TempCompass (Liu et al., 2024d) Temporal Understanding	43.1%	49.0%	53.7%	53.4% InternVL2-2B
Average	Across Benchmarks	44.0%	51.0%	59.8%	–
RAM Usage	Batch size = 1	0.8 GB	1.2 GB	4.9 GB	27.7 GB MolmoE-A1B-7B
	batch size = 64	15.0 GB	16.0 GB	49.9 GB	–

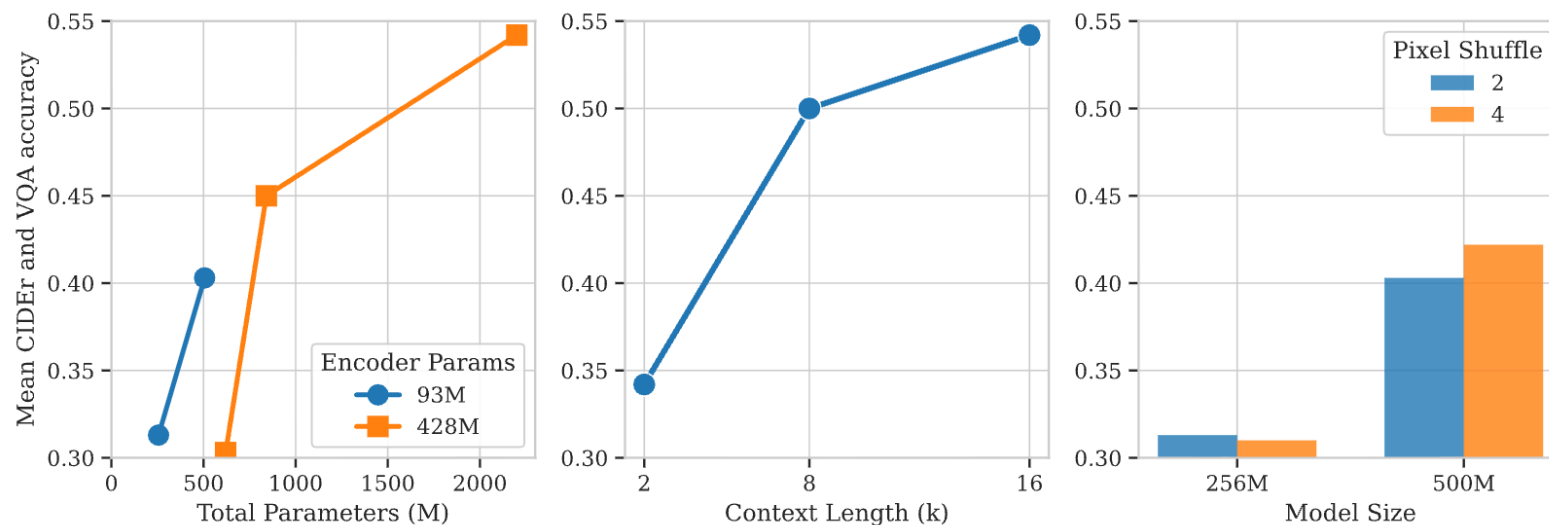
SmolVLM – Large Scale Evaluation using VLMEvalKit

- For the evaluation, the authors use the VLMEvalKit that we saw in the previous lecture to compare against different evaluation benchmarks and different state-of-the-art models
- SmolVLM demonstrates remarkable computational efficiency compared to significantly larger models. Single-image inference requires only 0.8GB of VRAM for the 256M variant, 1.2GB for the 500M, and 4.9GB for the 2.2B—dramatically lower than the 27.7GB required by MolmoE-A1B-7B.
- Intriguingly, visually oriented tasks such as OCRBench also benefit markedly from scaling language model capacity, with a nearly 10-point improvement when moving from 256M (52.6%) to 500M (61.0%)

Marafioti, Andrés, et al. "Smolvlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

Some additional results

- The SmolVLM authors investigate some important aspects of the design of VLMs such as the effect of the visual encoder size, the context length, and the effect of pixel shuffle
- A larger vision encoder has a bigger impact when using a larger LLM backbone
- Context length plays an important role in evaluations due to the ability to encoder larger images or longer documents



Marafioti, Andrés, et al. "Smolvlm: Redefining small and efficient multimodal models." arXiv preprint arXiv:2504.05299 (2025).

SmolVLM is versatile for different downstream tasks

- **ColSmolVLM** utilizes the smaller SmolVLM variants (256M and 500M parameters) designed explicitly for on-device deployment
- **Smol Docling** is an ultra-compact 256M-parameter variant of SmolVLM, optimized explicitly for end-to-end multimodal document conversion tasks
- **BioVQA** leverages SmolVLM's compact and efficient architecture to address visual question answering tasks within the biomedical domain

And many more that you could create for **your own** MSc thesis projects 😊

SmolVLM – On Edge Devices

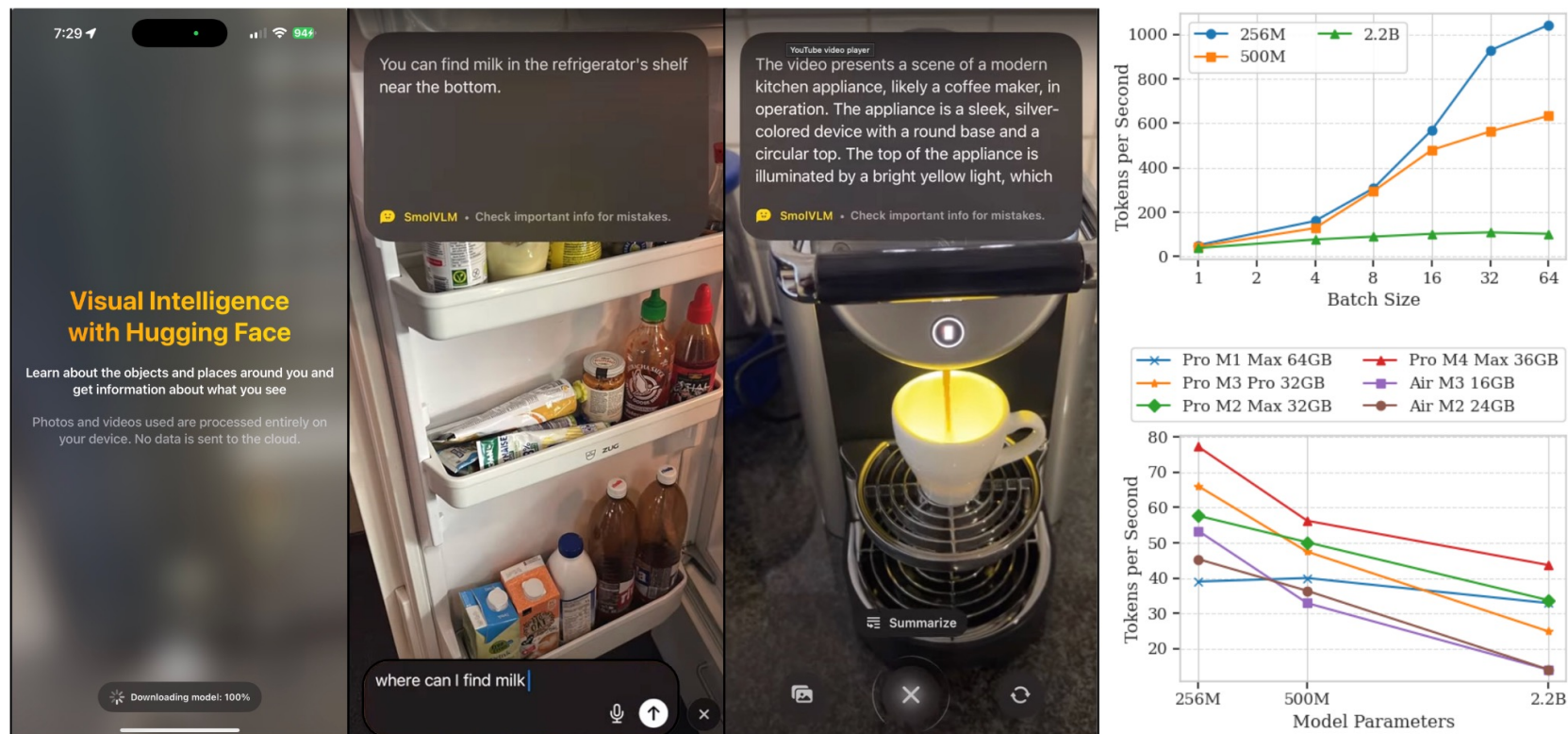
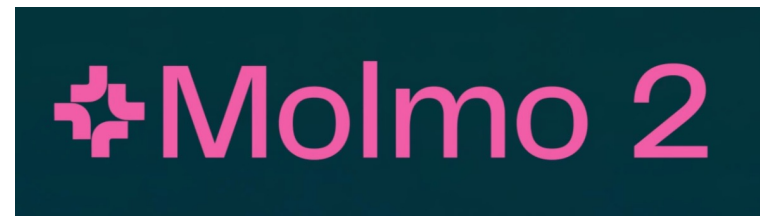


Figure 9 | SmolVLM on edge device. (Left) Examples of the [HuggingSnap](#) app, where SmolVLM can run locally, on the device, on consumer phones. For example, interactions can be done using a mobile interface to detect objects and answer questions. (Right) Throughput in tokens per second on NVIDIA A100 GPUs (top) and different consumer personal computers (bottom) across different batch sizes and model variants.

Other State-of-the-art VLMs



Gemma 3



LLaVA-OneVision

And many more...

Using VLMs via Ollama

Ollama offers a simple way to run many of these VLMs locally on your computer!

I have created a small tutorial that showcases how to use it and you can find it here:

<https://github.com/aleSuglia/ollama-tutorial>

Exciting avenue for VLM Research

- Fine-grained visual understanding via object-centric representations
- Versatile visual representations that operate at different levels of granularity (e.g., images, multi-image, videos, etc.)
 - How do we use a VLM to power a robot?
- Detailed study of the internal representations of a VLM
 - Is it really a Vision+Language model? Or more a *Language with some vision* model?
 - What happens to the model's internal representations after training?
 - How do model acquire concepts that they have never seen before?
- How do we build truly multimodal models for the real world?

Summary

- In today's lecture we saw how to implement frontier Vision and Language models that can be applied to a variety of different tasks
- We analysed the training pipeline for training visual prefix-based VLMs with a focus on SmolVLM, a state-of-the-art Vision and Language model created by Huggingface
 - Trained on open-source data
 - It can be downloaded from Huggingface
 - Thanks to its small memory footprint, it can be used for many difficult application domains
- In visual prefix-based, we saw that the number of tokens allocated for the visual modality has an effect on the accuracy of the models; important trade-off between computational accuracy and accuracy
- Creating Vision and Language models requires careful design of the training pipeline to take advantage of the benefits of pretrained vision and language models without catastrophic forgetting of their initial capabilities

Homework

- Revise this lecture slides as well as the previous background lecture to understand how SmolVLM relates to the broader literature on Multimodal Generative AI
- More details on SmolVLM can be found on the original paper: <https://arxiv.org/abs/2504.05299>; the paper contains many more experimental details that can help you understand the capabilities and design choices behind these models
- The blogpost is also another interesting resource if you want to know more: <https://huggingface.co/blog/smolvlm2>
- If you're really interested in understanding how these models are implemented, you should have a look at the source code available on Huggingface: <https://github.com/huggingface/transformers/tree/main/src/transformers/models/smolvlm>
- Finally, there are other important resources that you can consult if you want to work on Vision and Language Models:
 - <https://huggingface.co/blog/vlms>
 - <https://huggingface.co/blog/vlms-2025>
 - Or feel free to talk to me 😊